



Spatio-Cohesive Service Selection Using Machine Learning in Dynamic IoT Environments

KyeongDeok Baek^(✉) and In-Young Ko

School of Computing, Korea Advanced Institute of Science and Technology,
Daejeon, Republic of Korea
{kyeongdeok.baek,iko}@kaist.ac.kr

Abstract. In an Internet of Things (IoT) environment, a user may need to receive a set of services from the environment to accomplish his or her own task. Then, appropriate IoT devices that are necessary to provide the services should be located and accessed in a way that the functional effects of the IoT services can be delivered effectively to the user. Especially, it is more challenging to ensure the successful delivery of service effects when users and/or IoT devices have mobility in the environment. In this paper, we extend our previous work of spatially-cohesive service selection by adopting a reinforcement learning technique to effectively deal with highly dynamic situations in mobile IoT environments. We propose a reinforcement learning agent that selects services in terms of associating IoT devices in a dynamic manner while optimizing metrics such as spatio-cohesiveness and number of hand-overs during the period of providing the services. Our approach is evaluated by simulating the agent, and the results show that the agent successfully learns the optimal policy of the service selection.

Keywords: Spatio-cohesive service selection · Internet of Things
Reinforcement learning · Task-oriented computing

1 Introduction

The Web is now evolving to provide and connect not only the Web services delivered from traditional software components, but also the Internet of Things (IoT) services that can be provided by using a number of devices deployed over a dynamic environment such as a smart city. Therefore, there are new challenges of revisiting and extending existing techniques of service-oriented computing to be more scalable and efficient for dynamic IoT environments [1], such as service selection. Service selection problem is about selecting appropriate service instances among discovered candidates from various service providers in terms of non-functional requirements [2]. Especially in an IoT environment, when a user sends requests for a set of services to accomplish own task, the appropriate IoT devices that are necessary to provide the services should be located and accessed.

For successful delivery of service effects such as media content displayed on monitors to the users and for efficient communication, the associating IoT devices should be located in a spatially cohesive manner [3]. In addition, it is crucial to maintain the quality of delivering service effects in a mobile environment.

In our previous work [3], we proposed spatio-cohesive service discovery and hand-over algorithms that discover and select services in a spatially cohesive manner, and dynamically performs service hand-over. The service hand-over is a concept of performing service selection dynamically and migrate service instances from device to device, to allow users to consume the services in a continuous manner even though the availability of service ctuates. We proposed a metric that measures how the selected services are located in a spatially cohesive manner in terms of the associating IoT devices. However, the algorithms were designed based on heuristics of greedy choices on the services which are associated to the nearest IoT devices, so make decisions short-sighted.

To deal with the limitations of our previous work, we adopt *reinforcement learning*, which is a class of simulation-based machine-learning techniques. Reinforcement learning is about training a smart agent that learns the optimal policy of taking actions in a certain type of environments to achieve a given goal, by simulating a number of trials and errors. In this paper, we formulate the spatio-cohesive service selection problem as a reinforcement learning problem by representing the selection process as a Partial-observable Markov Decision Process. Then, we use the actor-critic algorithm [4] to solve the problem.

Main advantages of applying reinforcement learning to the problem are: (1) maximization of long-term rewards, (2) robustness, and (3) model-free learning. First, our service selection agent learns to optimize metrics such as the spatio-cohesiveness in a long-term manner rather than considering only current snapshot of the environment, which is challenging in highly-dynamic IoT environments. Second, our agent is robust, which means that the service selection can be done even though the environment is observed partially, or encounters unexpected situations such as failures. Third, our agent learns to optimize metrics without knowing the internal model of environments, which result in a great efficiency to deal with heterogeneous IoT environments and potential to extend.

We evaluated our approach to the spatio-cohesive service selection problem by simulating the selection processes of the service selection agent we propose. We compared our approach with baseline algorithms that are based on the algorithm from our previous work. Consequently, we observed the results that show our service selection agent successfully learning the optimal policies for selection in terms of spatio-cohesiveness and the number of hand-overs.

Rest of this paper is organized as follow. In Sect. 2, we state related works. In Sect. 3, we formulate the problem as a reinforcement learning problem and explain our service selection agent. Then, in Sect. 4, we evaluate our approach by conducting simulations, followed by the conclusion in Sect. 5.

2 Related Work

There is a comparative survey on service selection for Web service composition [2]. The service selection problem can be transformed as an optimization problem in the case when Quality of Service (QoS) profiles of each provider is known and fixed. Most of the optimization approaches require full information of the environment, and enough computation power along with time to process optimization algorithms. However, a user in IoT environments only can observe partial environment, usually the vicinity of the user, and devices have limited resources that might not be sufficient to run such optimization algorithms. In this paper, we deal with the limitations by using reinforcement learning techniques.

There is a series of works done on solving adaptive service composition problem by using multi-agent reinforcement learning [5]. In the works, service composition processes are formulated as variations of Markov Decision Process, and perform multi-agent reinforcement learning on the processes. However, their work focuses mostly on finding the optimal solution for a specific service composition process, whereas our work targets on the general spatio-cohesive service selection processes. Therefore, their service composition agents trained for a certain process cannot be applied to other service composition processes.

3 Machine Learning for Spatio-Cohesive Service Selection

Figure 1 shows the overview of our service selection agent. The agent on a user's mobile device observes IoT devices (service providers) in the surrounding environment. Based on the observation, the agent classifies service providers into certain categories of service types. We assume that exclusive classification of service functionalities is supported within a fixed number of service types. For each necessary service type, the agent selects an IoT device by applying a policy to the candidate service providers, which is implemented as neural networks. Finally, selections for each service type form a joint selection as the action that the agent performs at the time. As a feedback of the action, the agent gains rewards from the environment and updates its policy to maximize the cumulative reward, which is based on the spatio-cohesiveness and the number of hand-overs.

In this paper, we formulate the spatio-cohesive service selection process as a *Partial-Observable MDP* (POMDP), which is an extension of *Markov Decision Process* (MDP) where an agent has partial-observability on the environment. A POMDP is defined as a tuple: $\langle \mathcal{S}, \mathcal{A}, O, T, r \rangle$. $\mathcal{S}$ is a set of *states* an agent can have. $\mathcal{A}$ is a set of *actions* an agent can take to transit to another state. O is an observation function which provides an agent partial observation on the state, $O(s)$. T is a transition function, which is the probability of transition to another state after an agent takes an action. r is a reward function that generates a reward for each transition to either reinforce or discourage the action, $r(s, a, s')$.

3.1 Problem Definition

Our target environment is a two-dimensional space where a number of IoT devices are deployed as service providers, and there is a user agent that wants

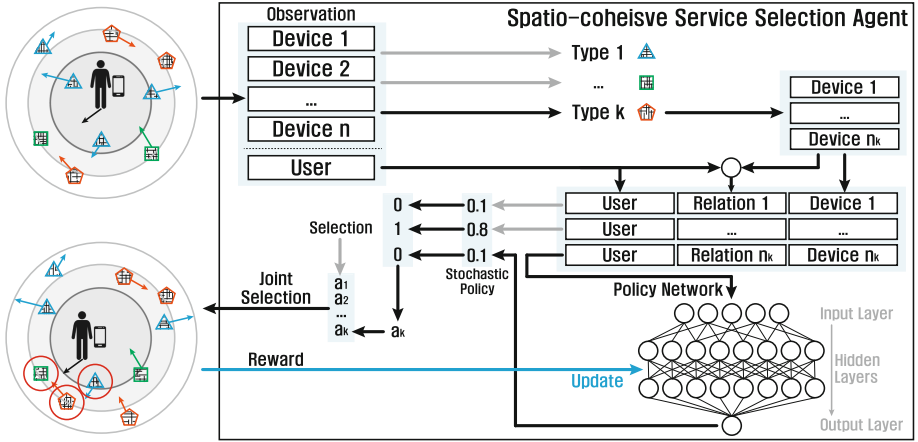


Fig. 1. Overview of the spatio-cohesive service selection process by the agent

to perform a task by utilizing the services in the environment. We call such IoT devices and user agents as *objects* in this problem definition. We assume that each object knows its coordinate and velocity in the space as a property. We represent a state of each device and user agent as a vector that is composed of information such as coordinate, speed, and service type. Finally, the aggregation of the states of objects in an environment becomes the state of the environment.

In our problem space, the agent has partial-observability in terms of Euclidean distances between objects, so the observation on services is restricted to the vicinity of the user within a predefined observation-range. At each time, t , the agent, u , gain partial observation, $O(u|s_t)$, on the current state s_t , which is a set of states of available IoT devices in the observation range.

The actions are the behaviors of selecting service providers among the candidates, where a single service provider is selected for each service type. In our problem, the number of candidate service providers varies according to available IoT devices. For this reason, we adopt the policy gradient method among the reinforcement learning algorithms, which is known to be more robust to deal with action spaces that are non-deterministic and large [6].

We define two reward metrics: spatio-cohesiveness, and the number of service hand-overs. First, we define *spatio-cohesiveness* as the sum of the normalized distance between the user and the selected service providers. Second, we define *the number of hand-overs* as a size of set difference between the previous and current set of selected services. We subtract the number of hand-overs from spatio-cohesiveness to generate a single reward function, to maximize spatio-cohesiveness and minimize the number of hand-overs.

3.2 Service Selection Agent

Our agent maintains stochastic policies, π_k , for each required type of service, k , that are defined as probabilities to select a service provider, p_k , under given observation, i.e. $\pi_k(p_k|o_k, \theta_k) = \mathbb{P}[p_k|o_k, \theta_k]$. The policies form a joint policy, π , which is a joint probability that is defined as the following Eq. 1:

$$\pi(p|o, \theta) = \mathbb{P}[p|o, \theta] = \prod_k \mathbb{P}[p_k|o_k, \theta_k] = \prod_k \pi_k(p_k|o_k, \theta_k), \quad (1)$$

where o and θ are the joint observation and the parameter, respectively. We assume that policies are independent of each other because they make decisions individually based on exclusive observations. The gradient of the joint policy can be derived based on the *policy gradient theorem* [7], as the following Eq. 2:

$$\nabla_{\theta} \ln \pi(p|o, \theta) = \nabla_{\theta} \ln \prod_k \pi_k(p_k|o_k, \theta_k) = \sum_k \nabla_{\theta_k} \ln \pi_k(p_k|o_k, \theta_k). \quad (2)$$

As a result, the gradient to update the joint policy can be obtained by summing the gradients of individual policies.

To parametrize the policies, we use feed-forward neural networks, which are known to be the most effective technique for parameterizing stochastic policies. For each candidate service provider, the agent processes the corresponding vector as an input to the policy network. The input vector is constructed by concatenating states of the user, the candidate service provider, and the relationship between them. The relationship vector is obtained by taking the absolute value of the difference between the user and service provider state vectors, which represent the relative position and speed of the service provider to the user. For each service type, the agent collects input vectors of the candidate service providers. After feeding the input vectors to the policy network, the output layer produces the probability values of selecting for each service provider.

3.3 Training Algorithm

To train the agent, we use the actor-critic algorithm [4], which is one of the state-of-the-art algorithms among policy-gradient methods, as shown in Algorithm 1. The algorithm using two separated neural networks: an *actor* and a *critic*. The actor networks approximate policies and makes decisions, and the critic networks are supporting-networks that suggests the right direction to update parameters of the actor network. Initially, parameters of actor and critic networks are initialized randomly (Line 1). The agent experiences episodes of simulations and learns from the collected experiences (Lines 2–13). An episode starts with resetting the environment (Line 3) and receiving an initial observation from the environment (Line 4). In an episode, the agent iteratively simulates the selections for given observations (Line 7) if all the service types are available by at least one service provider (Line 6), and gains rewards as feedbacks to judge whether the selection was optimal or not (Lines 5–12). Then, the state of the environment is changed

according to the selection and internal dynamics of the environment, and the agent gains some rewards from the environment (Line 8). Finally, based on the experiences, the agent calculates the gradient for the actor and the critic, and updates the parameters toward optimum (Lines 9–10).

Algorithm 1. Actor-critic algorithm for training service selection agent

Require: environment, env

Require: the number of episodes and steps, $num_episodes, num_steps$

Require: learning rate for actor and critic, α_a, α_c

```

1:  $\theta \leftarrow initialize(), w \leftarrow initialize()$ 
2: for episode = 1 ...  $num\_episodes$  do
3:    $env.reset()$ 
4:    $observation \leftarrow env.get\_observation()$ 
5:   for step = 1 ...  $num\_steps$  do
6:     if feasible( $observation$ ) then
7:        $selection \sim \pi(\cdot | observation, \theta)$ 
8:        $new\_observation, reward \leftarrow env.action(selection)$ 
9:        $w \leftarrow w + \alpha_c \times \text{gradient\_critic}(observation, reward, next\_observation)$ 
10:       $\theta \leftarrow \theta + \alpha_a \times \text{gradient\_actor}(observation, reward, next\_observation)$ 
11:     end if
12:   end for
13: end for

```

4 Evaluation

4.1 Simulation Setting

To evaluate our approach, we performed several simulations of the service selection agent in an IoT environment where a user and a number of IoT devices are deployed randomly. We implemented the agent by using TensorFlow¹, which is the most popular framework for machine-learning. The simulations were performed on a Windows 10 Pro machine within an Intel i7-3770 3.40 GHz CPU, 16 GB RAM, and nVidia GeForce GTX 1070 GPU.

In the simulation environment, total 1000 IoT devices and a user were deployed in a uniformly random manner over a 100 m $\times$ 100 m rectangular area. The number of IoT devices is empirically decided such that the distribution of the devices is not too sparse to make the service selection realistic. A service type is assigned randomly for each IoT device out of five service types. The user has 10 m of observation range, which is similar to Wi-Fi range, while the objects has mobility of a random speed, and some devices that have low speed are regarded as semi-static devices. The limit of maximum speed is set differently for each

¹ <https://www.tensorflow.org/>.

simulation. For better visualization, we fit the results with polynomial regression of 10th degree and scale noises by 20% according to the regression curve, by using SciPy², which is the most popular library used for scientific computations.

We compare our service selection approach against two baseline algorithms and a random selection algorithm. The baseline algorithms are designed based on the heuristics of our previous work [3], while one focuses on maximizing the spatio-cohesiveness (baseline-nearest) and the another focuses on reducing the number of hand-overs (baseline-hand-over).

4.2 Results and Analysis

Figure 2 shows the average reward gained for each episode of simulation with setting the maximum speed of objects as 4 m/s. We set the speed range to simulate static devices, pedestrians, and low-speed vehicles. Total 1000 episodes were conducted, and the average rewards are bounded from -2.7 to 0.5 , that means the algorithms suffer from many hand-overs caused by the high mobility. The result shows that our approach performs poorly in early episodes, but the performance increases steadily as our agent learns from experiences, and finally exceeds the performance of the baselines at around the 250th episode. Therefore, we can conclude that our agent successfully learns the optimal policy.

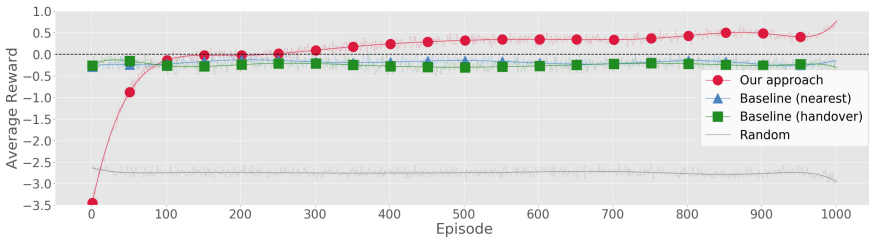


Fig. 2. Average reward compared to baseline approaches

Figure 3 also shows the average reward gained for each episode but setting the maximum speed of objects from 2 m/s to 5 m/s. The result shows that the average reward decreases as the maximum speed increases due to more dynamic nature of the environment. When the maximum speed is set as 2 m/s, there is not much difference of the rewards between our approach and the baseline approaches. However, the difference become larger as the maximum speed increases. This means that our approach is more robust and is capable of dealing with highly dynamic IoT environments in an efficient manner.

The size of our simulation environment is large enough to simulate practical IoT environments, because users usually interact with only the IoT devices that are located in their vicinity even in a large-scale environment.

² <https://www.scipy.org/>.

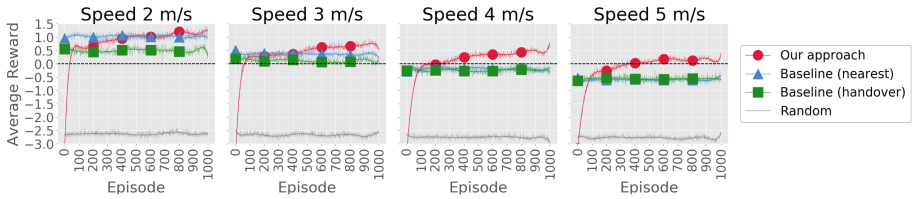


Fig. 3. Average reward within different maximum speed of objects

5 Conclusion

To perform user tasks in an effective and continuous manner, the service selection in IoT environments needs to be performed by choosing spatially cohesive service providers and making service provision robust, even though the availability and QoS of associated IoT devices change. In this paper, to overcome the limitations of our previous work, we propose a service selection method that utilizes reinforcement learning techniques that performs service selection in a predictive manner. Our approach is capable of optimizing the spatio-cohesiveness and the number of hand-overs in a long-term period of executing user tasks.

We evaluated our approach by conducting a number of simulations. The results show that the policy learned by our service selection agent converges to an optimal policy, which makes an efficient trade-off between the spatio-cohesiveness and the number of hand-overs, and also show that our agent performs in a stable manner even when the mobility of the user and IoT devices is high.

The main contribution of this work is in formulating and solving the spatio-cohesive service selection problem as a reinforcement learning problem, especially in a form of POMDP. Furthermore, even though current model of environments and optimization goals are quite limited, our approach has great potential of extending to more realistic and practical environments. In our future work, we plan to extend our agent considering inter-relationship among IoT devices and the situation where multiple users need to share services in a local environment.

Acknowledgement. This research was supported by Next-Generation Information Computing Development Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science, ICT (NRF-2017M3C4A7066210).

References

1. Issarny, V., Bouloukakakis, G., Georgantas, N., Billet, B.: Revisiting service-oriented architecture for the IoT: a middleware perspective. In: Sheng, Q.Z., Stroulia, E., Tata, S., Bhiri, S. (eds.) ICSOC 2016. LNCS, vol. 9936, pp. 3–17. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46295-0_1
2. Moghaddam, M., Davis, J.G.: Service selection in web service composition: a comparative review of existing approaches. In: Bouguettaya, A., Sheng, Q., Daniel, F. (eds.) Web Services Foundations, pp. 321–346. Springer, New York (2014). https://doi.org/10.1007/978-1-4614-7518-7_13

3. Baek, K.-D., Ko, I.-Y.: Spatially cohesive service discovery and dynamic service handover for distributed IoT environments. In: Cabot, J., De Virgilio, R., Torlone, R. (eds.) ICWE 2017. LNCS, vol. 10360, pp. 60–78. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-60131-1_4
4. Konda, V.R., Tsitsiklis, J.N.: Actor-critic algorithms. In: NIPS (2000)
5. Ren, L., Wang, W., Xu, H.: A reinforcement learning method for constraint-satisfied services composition. *IEEE Trans. Serv. Comput.* (2017)
6. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning (2015)
7. Sutton, R.S., Mcallester, D., Singh, S., Mansour, Y.: Policy gradient methods for reinforcement learning with function approximation. In: *Advances in Neural Information Processing Systems*, vol. 12, pp. 1057–1063 (1999)